Dual OS1 Calibration Pipeline Notes

Last updated: 2026-09-01

This document summarizes the current dual-Ouster calibration workflow for `~/drone_ws`: how the two calibration nodes work, the full operating order, and the terminal commands to run from a clean start.

1. What We Are Calibrating

The tracker needs one 6-DoF pose for each LiDAR:

```
lidar1_x: ...
lidar1_y: ...
lidar1_z: ...
lidar1_roll: ...
lidar1_pitch: ...
lidar1_yaw: ...

lidar2_x: ...
lidar2_y: ...
lidar2_z: ...
lidar2_roll: ...
lidar2_pitch: ...
lidar2_yaw: ...
```

The final tracker transform is:

```
P_world = Rz(yaw) * Ry(pitch) * Rx(roll) * P_sensor + [x, y, z]
```

The calibration is intentionally split into two stable subproblems:

```
ground_plane_node
  -> z, roll, pitch

sensor_calibration_node
  -> x, y, yaw
```

This is not a full 3D nonlinear 6-DoF optimization. It is a practical field method: the ground plane strongly constrains height/tilt, while surveyed grid targets strongly constrain planar position/yaw.

2. Code 1: ground_plane_node

File:

```
src/grid_sensor_calibration/grid_sensor_calibration/ground_plane_node.py
```

Purpose:

```
Measure LiDAR z, roll, pitch from the ground plane.
```

Principle:

1. The LiDAR sees the ground as many points near `sensor_z = -lidar_height`.
2. The node crops likely ground points using:
   - expected height band
   - range ring

- optional azimuth window

3. It fits a plane with RANSAC.
4. It refines the plane with SVD on the inliers.
5. It converts the plane into:
   - `z`: distance from sensor origin to ground
   - `roll`: tilt around sensor x
   - `pitch`: tilt around sensor y
6. It repeats over multiple frames and reports the median.

Plane equation:

```
n . p + d = 0
```

Meaning:

```
lidar_z = d
pitch = asin(-n_x)
roll  = atan2(n_y, n_z)
```

Quality indicators:

```
height spread      small is good
roll/pitch spread  small is good
residual sigma     10-20 mm is good
inlier fraction    >70% is preferred
downhill azimuth   should make geometric sense
```

Important:

`expected_height` is only used to crop the likely ground band. It does not need to be exact, but it should be close enough that the ground survives the crop.

For 2 m LiDAR installation, use `expected_height:=2.0`.

3. Code 2: sensor_calibration_node

File:

```
src/grid_sensor_calibration/grid_sensor_calibration/sensor_calibration_node.py
```

Purpose:

```
Measure LiDAR x, y, yaw from known grid target locations.
```

Input:

```
known grid point:  (X, Y)
measured target:   sensor point (x, y, z)
```

The node solves:

```
grid_xy = Rz(yaw) * corrected_sensor_xy + [x, y]
```

Current improvement:

The node can now use the `z/roll/pitch` found by `ground_plane_node`:

```
-p apply_tilt_correction:=true
-p sensor_z:=...
```

```
-p sensor_roll:=...
-p sensor_pitch:=...
```

When enabled, captured `(x, y, z)` points are corrected before the planar fit using the known roll/pitch:

```
P_tilt = Ry(pitch) * Rx(roll) * P_sensor
corrected_sensor_xy = P_tilt.xy
```

How the three ground-plane values are used:

```
z/roll/pitch are all used by the final tracker 3D transform.

For the planar x/y/yaw solve, roll/pitch are also used immediately to
tilt-correct the captured sensor xyz points before fitting.

sensor_z is carried through as part of the same ground-plane calibration set
and is printed in the paste-ready YAML block. It is not directly added inside
the 2D x/y/yaw solve; the z translation is applied later by the tracker.
```

The capture file still stores original raw `(x, y, z)`. The correction is only applied when solving, so the same captures can be recalculated with correction on/off.

The `run_calibration` and `status` outputs now include a paste-ready YAML block:

```
lidar1_x:      1.585
lidar1_y:      3.401
lidar1_z:      0.896
lidar1_roll:   0.004264
lidar1_pitch:  0.000229
lidar1_yaw:    1.528751  # 0.486710 * pi
```

YAML does not evaluate `pi`; paste the numeric radian value before the comment.

4. Code 3: pointcloud_target_node

File:

```
src/grid_sensor_calibration/grid_sensor_calibration/pointcloud_target_node.py
```

Purpose:

```
Extract one calibration target center from a PointCloud2 stream and publish it
as PointStamped.
```

Flow:

```
/lidarN/points
  -> sensor-frame box crop
  -> optional voxel downsample
  -> DBSCAN clusters
  -> choose target cluster
  -> reduce cluster to one point
  -> /sensorN/target_point
```

Target reduction modes:

```
target_shape:=centroid
  Uses median of cluster points.

target_shape:=cylinder
  Fits a known-radius circle to the xy projection.
  Better for a vertical tube because it reduces one-sided-view bias.
```

For the real 24 x 15 field test, the planned calibration object is a paper

tube target:

```
diameter: 0.30 m
radius:   0.15 m
height:   0.015 m, if the stated 1.5 cm value is literal
```

Use:

```
-p target_shape:=cylinder
-p target_radius:=0.15
```

Important height note:

```
If the target height is really 1.5 cm, automatic extraction will be very hard.
The LiDAR will mostly see only a thin slice very close to the ground, and it can
be confused with floor/grass/ground points. In that case, click-based capture or
a taller vertical marker on top of the tube is safer.
```

For a 2 m LiDAR and a tall vertical target, a starting sensor-frame z box is:

```
-p 'box:=[-30.0,30.0,-30.0,30.0,-1.7,0.8]'
```

Explanation:

```
ground is near z=-2.0
start at -1.7 to remove ground
top of a tall 3 m target is near z=+1.0 relative to a 2 m sensor
0.8 is a conservative first upper bound for a tall marker
```

If it misses the target, widen the z range carefully. If it chooses people or walls, narrow the box or clear the field.

For a very low 1.5 cm target, the z box must include the ground-adjacent band:

```
-p 'box:=[-30.0,30.0,-30.0,30.0,-2.05,-1.80]'
```

This is not recommended unless the ground is clean and flat, because the target and the ground are almost the same vertical layer.

5. 24 x 15 Field Scenario

Assumed field:

```
flight area: x = 0..24, y = 0..15
safety zone: 4 m around all sides
main flight altitude: about 2 m
flight altitude range: 0..4 m
LiDAR height recommendation: 2 m
```

Recommended LiDAR placement:

```
lidar1: about (12, -3), z=2 m, facing +Y
lidar2: about (12, 18), z=2 m, facing -Y
```

Nominal yaw:

```
lidar1_yaw ~= +1.5708 rad
lidar2_yaw ~= -1.5708 rad
```

3 x 3 grid target points:

```
0 -> (3,3)
1 -> (12,3)
2 -> (21,3)
3 -> (3,9)
```

```
4 -> (12,9)
5 -> (21,9)
6 -> (3,15)
7 -> (12,15)
8 -> (21,15)
```

Flattened ROS parameter:

```
-p 'grid_points:=[3.0,3.0,12.0,3.0,21.0,3.0,3.0,9.0,12.0,9.0,21.0,9.0,3.0,15.0,12.0,15.0,21.0,15.0
    ]'
```

Why 9 points:

```
more points average random extraction/click error
wide baseline improves yaw accuracy
bad targets are easier to find from residuals
```

6. Full Terminal Workflow

6.1 Clean Start

```
pkill -f "ros2"
pkill -f "rviz2"
pkill -f "ptp4l"
pkill -f "os_driver"
pkill -f "lidar_drone_tracker"
pkill -f "sensor_calibration"
pkill -f "pointcloud_target_node"
pkill -f "competition_gui"
pkill -f "compet_score"
ros2 daemon stop
```

6.2 Terminal 1: PTP

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

sudo ptp4l -f ~/drone_ws/ptp_gm.conf -m
```

6.3 Terminal 2: Dual Ouster Driver

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 launch dual_ouster_driver dual_os1.launch.py
```

Check:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 topic hz /lidar1/points
ros2 topic hz /lidar2/points
```

Expected:

```
about 10 Hz per LiDAR
```

6.4 Terminal 3: Ground Plane for lidar1

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 run grid_sensor_calibration ground_plane_node --ros-args \
  -p input_topic:=/lidar1/points \
  -p sensor_label:=lidar1 \
  -p expected_height:=2.0 \
  -p num_frames:=20 \
  -p range_max:=20.0
```

If a fence, car, wall, or people are in one direction, restrict azimuth:

```
ros2 run grid_sensor_calibration ground_plane_node --ros-args \
  -p input_topic:=/lidar1/points \
  -p sensor_label:=lidar1 \
  -p expected_height:=2.0 \
  -p num_frames:=20 \
  -p range_max:=20.0 \
  -p azimuth_min:=-120.0 \
  -p azimuth_max:=120.0
```

6.5 Terminal 3: Ground Plane for lidar2

```
ros2 run grid_sensor_calibration ground_plane_node --ros-args \
  -p input_topic:=/lidar2/points \
  -p sensor_label:=lidar2 \
  -p expected_height:=2.0 \
  -p num_frames:=20 \
  -p range_max:=20.0
```

Record the printed YAML values:

```
lidar1_z:     ...
lidar1_roll:  ...
lidar1_pitch: ...

lidar2_z:     ...
lidar2_roll:  ...
lidar2_pitch: ...
```

These three values per LiDAR are used in the next calibration step.

6.6 Terminal 3: Target Extractor for lidar1

Use this if using automatic extraction:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 run grid_sensor_calibration pointcloud_target_node --ros-args \
  -p input_topic:=/lidar1/points \
  -p output_topic:=/sensor1/target_point \
  -p frame_id:=lidar1 \
  -p 'box:=[-30.0,30.0,-30.0,30.0,-1.7,0.8]' \
  -p cluster_tolerance:=0.20 \
  -p cluster_min_points:=8 \
  -p cluster_max_extent:=3.2 \
  -p selection:=largest \
  -p target_shape:=cylinder \
  -p target_radius:=0.15 \
  -p max_fit_residual:=0.04 \
  -p voxel_size:=0.02
```

6.7 Terminal 4: Target Extractor for lidar2

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 run grid_sensor_calibration pointcloud_target_node --ros-args \
  -p input_topic:=/lidar2/points \
  -p output_topic:=/sensor2/target_point \
  -p frame_id:=lidar2 \
  -p 'box:=[-30.0,30.0,-30.0,30.0,-1.7,0.8]' \
  -p cluster_tolerance:=0.20 \
  -p cluster_min_points:=8 \
  -p cluster_max_extent:=3.2 \
  -p selection:=largest \
  -p target_shape:=cylinder \
  -p target_radius:=0.15 \
  -p max_fit_residual:=0.04 \
  -p voxel_size:=0.02
```

Check target output:

```
ros2 topic echo /sensor1/target_point
ros2 topic echo /sensor2/target_point
```

6.8 Alternative: Clicked Point Instead of Extraction

Use clicked point for debugging or when extraction is unstable.

RViz:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 run rviz2 rviz2 -d ~/drone_ws/install/grid_sensor_calibration/share/grid_sensor_calibration/r
    viz/target_pick.rviz
```

Echo:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 topic echo /clicked_point
```

For clicked mode, use `input_topic:=/clicked_point` in the next node commands.

6.9 Terminal 5: sensor1 Calibration Node

Automatic target extraction input:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor1_calibration \
  -p sensor_name:=sensor1 \
  -p child_frame:=lidar1 \
  -p input_topic:=/sensor1/target_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=10 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[3.0,3.0,12.0,3.0,21.0,3.0,3.0,9.0,12.0,9.0,21.0,9.0,3.0,15.0,12.0,15.0,21.0,15
    .0]' \
  -p calibration_file:=$HOME/grid_calibration/field_24x15/sensors_calibration.yaml \
  -p nominal_x:=12.0 \
  -p nominal_y:=-3.0 \
  -p nominal_yaw:=1.5708 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=REPLACE_LIDAR1_Z \
  -p sensor_roll:=REPLACE_LIDAR1_ROLL \
  -p sensor_pitch:=REPLACE_LIDAR1_PITCH
```

Clicked point input:

```
ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor1_calibration \
  -p sensor_name:=sensor1 \
  -p child_frame:=lidar1 \
  -p input_topic:=/clicked_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=1 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[3.0,3.0,12.0,3.0,21.0,3.0,3.0,9.0,12.0,9.0,21.0,9.0,3.0,15.0,12.0,15.0,21.0,15
    .0]' \
  -p calibration_file:=$HOME/grid_calibration/field_24x15_click/sensors_calibration.yaml \
  -p nominal_x:=12.0 \
  -p nominal_y:=-3.0 \
  -p nominal_yaw:=1.5708 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=REPLACE_LIDAR1_Z \
  -p sensor_roll:=REPLACE_LIDAR1_ROLL \
  -p sensor_pitch:=REPLACE_LIDAR1_PITCH
```

6.10 Terminal 6: sensor2 Calibration Node

Automatic target extraction input:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor2_calibration \
  -p sensor_name:=sensor2 \
  -p child_frame:=lidar2 \
  -p input_topic:=/sensor2/target_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=10 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[3.0,3.0,12.0,3.0,21.0,3.0,3.0,9.0,12.0,9.0,21.0,9.0,3.0,15.0,12.0,15.0,21.0,15
    .0]' \
  -p calibration_file:=$HOME/grid_calibration/field_24x15/sensors_calibration.yaml \
  -p nominal_x:=12.0 \
  -p nominal_y:=18.0 \
  -p nominal_yaw:=-1.5708 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=REPLACE_LIDAR2_Z \
  -p sensor_roll:=REPLACE_LIDAR2_ROLL \
  -p sensor_pitch:=REPLACE_LIDAR2_PITCH
```

Clicked point input:

```
ros2 run grid_sensor_calibration sensor_calibration_node --ros-args \
  -r __node:=sensor2_calibration \
  -p sensor_name:=sensor2 \
  -p child_frame:=lidar2 \
  -p input_topic:=/clicked_point \
  -p input_message_type:=point_stamped \
  -p capture_sample_count:=1 \
  -p auto_calibrate:=false \
  -p 'grid_points:=[3.0,3.0,12.0,3.0,21.0,3.0,3.0,9.0,12.0,9.0,21.0,9.0,3.0,15.0,12.0,15.0,21.0,15
    .0]' \
  -p calibration_file:=$HOME/grid_calibration/field_24x15_click/sensors_calibration.yaml \
  -p nominal_x:=12.0 \
  -p nominal_y:=18.0 \
  -p nominal_yaw:=-1.5708 \
  -p apply_tilt_correction:=true \
  -p sensor_z:=REPLACE_LIDAR2_Z \
  -p sensor_roll:=REPLACE_LIDAR2_ROLL \
  -p sensor_pitch:=REPLACE_LIDAR2_PITCH
```

7. Capture Procedure

For the 9-point 24 x 15 grid, move the target in this order:

```
0 -> (3,3)
1 -> (12,3)
2 -> (21,3)
3 -> (3,9)
4 -> (12,9)
5 -> (21,9)
6 -> (3,15)
7 -> (12,15)
8 -> (21,15)
```

At each target:

1. Put the target exactly on the known grid point.
2. Confirm `/sensor1/target_point` and `/sensor2/target_point` are on the same physical target in RViz.
3. Capture both sensors:

```
ros2 service call /sensor1_calibration/capture_point std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/capture_point std_srvs/srv/Trigger
```

Check status:

```
ros2 service call /sensor1_calibration/status std_srvs/srv/Trigger
```

```
ros2 service call /sensor2_calibration/status std_srvs/srv/Trigger
```

If one target is bad, recapture only that index:

```
ros2 param set /sensor1_calibration capture_index 4
ros2 service call /sensor1_calibration/capture_point std_srvs/srv/Trigger

ros2 param set /sensor2_calibration capture_index 4
ros2 service call /sensor2_calibration/capture_point std_srvs/srv/Trigger
```

Reset all captures:

```
ros2 service call /sensor1_calibration/reset std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/reset std_srvs/srv/Trigger
```

Undo the last capture:

```
ros2 service call /sensor1_calibration/undo_point std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/undo_point std_srvs/srv/Trigger
```

8. Solve XY/Yaw

After all points are captured:

```
ros2 service call /sensor1_calibration/run_calibration std_srvs/srv/Trigger
ros2 service call /sensor2_calibration/run_calibration std_srvs/srv/Trigger
```

Expected output:

```
sensor1: x=..., y=..., yaw=... rad (... deg), rms=... m

paste into lidar_localization yaml:

lidar1_x: ...
lidar1_y: ...
lidar1_z: ...
lidar1_roll: ...
lidar1_pitch: ...
lidar1_yaw: ...  # ... * pi
```

Paste the numeric values into the tracker parameter YAML:

```
src/drone_localization/params/lidar_localization_os1.yaml
```

or the current practice YAML:

```
src/drone_localization/params/lidar_localization_4x4.yaml
```

YAML-only parameter changes do not require rebuilding. Restart the tracker to load the new values.

9. Validation Criteria

Ground plane:

```
residual sigma: 10-20 mm good
roll/pitch spread: as small as possible, ideally <0.1 deg
height spread: ideally cm-level
inlier fraction: >70% preferred
```

XY/yaw:

```
RMS: 2-5 cm good
specific target residual high: recapture that target
lidar1_yaw: near +90 deg for bottom sensor
lidar2_yaw: near -90 deg for top sensor
```

```
yaw difference: near 180 deg
```

Tracker:

```
/drone/estimated_pose near 10 Hz
Detection ratio high
LiDAR position disagreement warnings rare
filtered point clouds visible in RViz
```

10. Run Tracker After Calibration

Regular tracker:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 launch drone_localization lidar_localization.launch.py \
  params_file:=lidar_localization_os1.yaml \
  start_driver:=false \
  tracker_executable:=lidar_drone_tracker
```

EKF tracker:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 launch drone_localization lidar_localization.launch.py \
  params_file:=lidar_localization_os1.yaml \
  start_driver:=false \
  tracker_executable:=lidar_drone_tracker_ekf
```

RViz:

```
source /opt/ros/humble/setup.bash
source ~/drone_ws/install/setup.bash

ros2 run rviz2 rviz2 -d ~/drone_ws/install/dual_ouster_driver/share/dual_ouster_driver/rviz/tracke
    r_view.rviz
```

Check output:

```
ros2 topic hz /drone/estimated_pose
ros2 topic echo /drone/estimated_pose
```

11. Why This Split Calibration Is Acceptable

Question:

```
If z/roll/pitch and x/y/yaw are solved separately, can roll/pitch later distort
the XY/yaw result?
```

Answer:

```
Yes, wrong roll/pitch can hurt. That is why ground-plane fit quality must be
checked. But when the ground plane is good, the split is practical and stable.
```

The updated calibration node reduces the mismatch:

```
old:
  captured sensor (x,y,z) -> use x,y directly

new:
  captured sensor (x,y,z)
  -> apply known roll/pitch for planar calibration
  -> use corrected x,y

final tracker:
  uses x/y/z/roll/pitch/yaw together in the 3D transform
```

Risk scale for 2 m LiDAR height:

```
roll/pitch error 0.1 deg -> about 0.35 cm xy bias
roll/pitch error 0.3 deg -> about 1.0 cm xy bias
roll/pitch error 1.0 deg -> about 3.5 cm xy bias
```

So if ground-plane roll/pitch is stable within about 0.1 deg, the correction is usually safer than assuming `roll=0`, `pitch=0`.

12. Practice 3 x 2 Variant

Practice grid:

```
0 -> (0,0)
1 -> (0,2)
2 -> (3,2)
3 -> (3,0)
```

Flattened parameter:

```
-p 'grid_points:=[0.0,0.0,0.0,2.0,3.0,2.0,3.0,0.0]'
```

Nominal practice LiDAR poses:

```
lidar1: x=1.5, y=3.5, yaw about -90 deg or +90 deg depending orientation
lidar2: x=1.5, y=-1.5, yaw opposite
```

Use the same commands as the 24 x 15 workflow, replacing only:

```
grid_points
calibration_file path
nominal_x / nominal_y / nominal_yaw
tracker params_file
tracker ROI
```